Name	Likith Kumar
Email ID	likithkumarg425@gmail.com
Course Code	102505
Assignment Topic	Enterprise SaaS DevOps Capstone Project
Source Code Provided	YES
GIT URL	https://github.com/LikithKumar0112/Enterprise-SaaS-Capstone-Project.git

Private & Confidential – Do not duplicate or distribute without written permission"

Identified Mistakes and Areas for Improvement Enterprise SaaS DevOps Capstone Project Review

Problem Statement

The project aims to implement an enterprise-style DevOps platform for a Node.js SaaS application using AWS, Terraform, Docker, Kubernetes/EKS, Helm, GitHub Actions, security scanning, monitoring, autoscaling, and cost controls.

The repository contains a substantial implementation across application code, infrastructure, CI/CD workflows, Kubernetes manifests, Helm charts, monitoring configuration, testing, and documentation. The review below is based on the actual submitted files and configuration rather than only the README claims.

Key Observations

1. Application Development and Testing

Issue Identified

The application implementation is well structured and includes several enterprise-oriented features such as Redis caching, circuit breaker handling, request IDs, rate limiting, Prometheus metrics, structured logging, and graceful shutdown.

The project also includes unit, integration, and load/E2E test suites.

Observed Behavior

The application has:

- Unit tests using Jest and Supertest.
- Integration tests that interact with Redis.
- E2E/load tests for concurrent requests and response performance.
- Coverage thresholds configured in package.json.
- ESLint validation in CI.
- Prometheus metrics through prom-client.
- /health and /metrics endpoints.

Private & Confidential – Do not duplicate or distribute without written permission"

- Redis and external API failure handling.

The test setup is more complete than a basic Node.js application and demonstrates good coverage of application-level DevOps requirements.

Opportunity for Enhancement
The E2E tests are relatively lightweight. For example, the load test sends a fixed number of requests and checks average response time, but it does not provide sustained load, percentile latency, throughput, or failure-rate analysis.

The integration tests also depend on Redis being available, which is correctly handled in GitHub Actions but should be clearly separated from tests that can run completely independently.

Why This Matters
Enterprise applications need testing that validates not only whether endpoints work, but also how the application behaves under dependency failures and increased traffic.

Key Findings

- Unit testing: Implemented
- Integration testing: Implemented
- Load/E2E testing: Implemented
- Coverage thresholds: Implemented
- Linting: Implemented
- Redis failure handling: Implemented
- Advanced performance benchmarking: Partial

Suggested Improvements
Add more meaningful performance measurements such as p95/p99 latency, request throughput, and error percentage. Additional integration scenarios could also cover Redis unavailable/recovery behavior and circuit-breaker recovery.

2. Docker Containerization and Application Runtime

Private & Confidential – Do not duplicate or distribute without written permission"

Issue Identified

The Docker implementation is strong, but some runtime behavior could be aligned more closely with the overall security and production design.

Observed Behavior

The Dockerfile uses a multi-stage build:

- Production dependencies are installed separately.
- The runtime image uses Node.js Alpine.
- Development dependencies are excluded.
- npm, npx, and corepack are removed from the runtime image.
- A dedicated non-root user with UID 1001 is created.
- dumb-init is used as PID 1.
- A Docker health check calls /health.
- The container exposes port 3000.
- Linux capabilities are restricted at the Kubernetes level.

This is a solid containerization approach.

Opportunity for Enhancement

The Dockerfile contains comments explaining vulnerability remediation and package upgrades, but the long-term reproducibility of apk upgrade should be considered carefully because it can change packages independently of the original base image version.

Also, the project contains generated coverage and log files inside the application repository. These should normally be excluded from the Docker build context through .dockerignore and from source control where appropriate.

Why This Matters

A production container should be small, reproducible, secure, and contain only the files required at runtime.

Key Findings

- Multi-stage build: Implemented

Private & Confidential – Do not duplicate or distribute without written permission"

- Non-root runtime: Implemented
- Health check: Implemented
- Minimal production dependencies: Implemented
- Init process handling: Implemented
- Container security controls: Implemented
- Build-context hygiene: Can be improved

Suggested Improvements

Keep generated reports/logs outside the production build context and review image reproducibility around OS package upgrades. Continue enforcing image scanning in CI.

3. AWS Infrastructure and Terraform Implementation

Issue Identified

The Terraform implementation covers a large amount of AWS infrastructure, but some components are configured for the environment without being fully integrated into the application runtime.

Observed Behavior

Terraform provisions/configures:

- VPC across two Availability Zones.
- Public and private subnets.
- NAT Gateway.
- EKS cluster.
- Managed node groups.
- ECR repository.
- AWS Secrets Manager.
- IAM/IRSA roles.
- GitHub Actions OIDC.
- CloudWatch alarms.
- SNS notifications.
- AWS Budget.
- Optional ElastiCache Redis.
- Remote S3 Terraform state with DynamoDB locking.

Private & Confidential – Do not duplicate or distribute without written permission"

The ECR repository also uses immutable image tags and scan-on-push.

The infrastructure demonstrates a good understanding of Infrastructure as Code and AWS integration.

Opportunity for Enhancement

The configured Terraform backend references:

- capstone-tf-bucket
- terraform-state-lock

but those backend resources are not provisioned by the Terraform code itself. Therefore, a fresh environment requires those resources to exist before terraform init can successfully initialize the backend.

There are also saved Terraform plan files such as tfplan_destroy and tfplan_lbc included in the repository. These are generated artifacts and should generally not be committed.

Why This Matters

Infrastructure automation should be reproducible from a clean environment, while state and generated plan files should be handled carefully because Terraform plans can contain sensitive infrastructure information.

Key Findings

- VPC: Implemented
- EKS: Implemented
- ECR: Implemented
- IAM/OIDC: Implemented
- Remote Terraform state: Configured
- Budget monitoring: Implemented
- ElastiCache option: Implemented but disabled
- Backend bootstrap: Requires external/pre-existing resources
- Terraform plan artifact management: Needs improvement

Suggested Improvements

Private & Confidential – Do not duplicate or distribute without written permission"

Document the bootstrap requirement for the S3 backend and DynamoDB lock table, or maintain a separate bootstrap configuration for them. Add generated Terraform plan files to .gitignore and remove them from the repository.

4. Kubernetes, Helm, Autoscaling, and High Availability

Issue Identified

The Kubernetes and Helm configuration is extensive and includes Deployment, Service, Ingress, HPA, PDB, NetworkPolicy, Redis, probes, and security contexts. However, there is an important Redis architecture concern and some duplicated configuration.

Observed Behavior

The Helm application provides:

- Configurable replica counts.
- CPU/memory requests and limits.
- Readiness and liveness probes.
- HPA.
- PodDisruptionBudget.
- ALB Ingress.
- Non-root execution.
- Dropped Linux capabilities.
- Seccomp RuntimeDefault.

The separate Kubernetes HPA supports both CPU and memory metrics, while the Helm HPA currently focuses on CPU.

The Redis deployment runs 2 replicas, but both pods are independent Redis instances. There is no Redis replication, Sentinel, or clustered Redis configuration connecting the two instances.

Opportunity for Enhancement

Running two independent Redis pods behind a single Kubernetes Service does not provide true Redis high availability.

Private & Confidential – Do not duplicate or distribute without written permission"

Requests may reach different Redis instances, resulting in inconsistent cache contents between pods.

The application is using Redis as a shared cache, so this behavior should be addressed if the intention is to demonstrate a highly available Redis architecture.

Why This Matters

Scaling an application and scaling its dependencies are different problems. Multiple Redis pods do not automatically provide shared state or failover.

Key Findings

- Kubernetes Deployment: Implemented
- Helm chart: Implemented
- HPA: Implemented
- PDB: Implemented
- Readiness/Liveness probes: Implemented
- NetworkPolicy: Implemented
- ALB Ingress: Implemented
- Application scaling: Implemented
- Redis high availability: Partial

Suggested Improvements

For a true HA design, use Amazon ElastiCache/managed Redis or configure Redis replication and failover appropriately. Also consider consolidating the standalone HPA/PDB configuration with the Helm-managed resources to avoid maintaining duplicate definitions.

5. CI/CD Pipeline and Deployment Strategy

Issue Identified

The CI pipeline is well developed, but the production CD workflow contains an important environment configuration issue.

Observed Behavior

Private & Confidential – Do not duplicate or distribute without written permission"

CI includes:

- Node.js setup.
- npm ci.
- ESLint.
- Unit tests.
- Integration tests.
- Redis service container.
- Snyk dependency scanning.
- Docker image build.
- Trivy image scanning.
- Hadolint Dockerfile scanning.

The development CD pipeline:

- Uses GitHub OIDC.
- Logs into ECR.
- Builds and pushes a commit-SHA image.
- Deploys through Helm.
- Waits for rollout.
- Performs rollback if rollout fails.

However, the production job currently uses:

- enterprise-devops-app-development as the ECR repository.
- ${{ vars.EKS_CLUSTER_NAME_DEV }} for the cluster.
- No values-prod.yaml during the production Helm deployment.

So although the workflow is named deploy-production, the actual configuration points to development resources.

Opportunity for Enhancement

The production deployment should have its own ECR repository/environment configuration, production EKS cluster variable, and production Helm values.

The current configuration could unintentionally deploy the main branch to the development environment rather than a separate production environment.

Private & Confidential – Do not duplicate or distribute without written permission"

Why This Matters

Environment isolation is one of the most important parts of an enterprise CI/CD design. Development and production should not share deployment targets unintentionally.

Key Findings

- CI pipeline: Implemented
- Automated development deployment: Implemented
- GitHub OIDC: Implemented
- ECR image publishing: Implemented
- Helm deployment: Implemented
- Rollback mechanism: Implemented
- Production approval: Configured
- Production target isolation: Needs correction
- Snyk blocking enforcement: Partial, because continue-on-error: true is configured.

Suggested Improvements

Create explicit production variables/repositories and deploy using values-prod.yaml. Remove continue-on-error if Snyk is intended to be a mandatory deployment gate.

6. Security, Secrets Management, and Identity

Issue Identified

The project includes several good security mechanisms, but the Secrets Manager and IRSA implementation is not fully connected to the application deployment.

Observed Behavior

Security controls include:

- GitHub Actions OIDC.
- IAM roles.
- IRSA configuration.
- AWS Secrets Manager.
- Trivy.
- Snyk.

Private & Confidential – Do not duplicate or distribute without written permission"

- Gitleaks.
- Non-root containers.
- Dropped capabilities.
- Seccomp.
- Kubernetes NetworkPolicy.
- ECR image scanning.

However, Terraform creates the application IAM role with a trust relationship for the service account:
default:enterprise-devops-app

but the Helm Deployment does not define a corresponding serviceAccountName.

Also, the Redis secret is created in AWS Secrets Manager, but the application Deployment does not retrieve that secret and inject it into the application. The Helm configuration simply uses:
REDIS_URL=redis://redis-service:6379

The Redis configuration itself does not use the generated authentication token.

Why This Matters
Creating a secret and IAM role is not sufficient by itself. The complete security flow should be:
Secret → IAM/IRSA → Kubernetes workload → Application → Dependency

At present, the infrastructure pieces exist, but the runtime integration is incomplete.

Key Findings

- GitHub OIDC: Implemented
- IAM/IRSA infrastructure: Implemented
- Secrets Manager: Implemented
- Application service-account integration: Missing
- Runtime secret injection: Missing
- Non-root containers: Implemented

Private & Confidential – Do not duplicate or distribute without written permission"

- NetworkPolicy: Implemented
- Security scanning: Implemented

Suggested Improvements

Create the Kubernetes ServiceAccount with the required IAM role annotation and configure the application to retrieve/use the Redis credential securely. Avoid creating security infrastructure that is not actually consumed by the workload.

7. Monitoring, Observability, Resilience, and Cost Optimization

Issue Identified

The project covers a broad observability and operational scope, including Prometheus, Grafana, CloudWatch, application metrics, alerts, HPA, logging, and cost controls. Some components, however, are more documented/configured than fully integrated.

Observed Behavior

The application exposes:

- Request counters.
- Request-duration histograms.
- Default process metrics.
- Structured Winston logs.
- Health endpoint.

The project includes:

- ServiceMonitor for Prometheus.
- Grafana dashboards/screenshots.
- CloudWatch alarms.
- SNS notifications.
- AWS Budget.
- ECR lifecycle policy.
- SPOT worker nodes.
- Single NAT Gateway.
- HPA.
- PDB.

Private & Confidential – Do not duplicate or distribute without written permission"

The cost report also documents the estimated running cost and teardown strategy.

One area requiring attention is the CloudWatch logging configuration. Terraform creates a log group under an /aws/ecs/... name, while the application is running on EKS, and there is no clear log shipping configuration connecting the application's container logs to that CloudWatch log group.

The chaos-engineering document is also incomplete. It contains a network-latency example, but it does not demonstrate a complete controlled experiment against the actual application.

Why This Matters

Observability should provide an end-to-end path from application telemetry to the monitoring platform. Similarly, chaos testing should demonstrate an actual failure scenario, expected behavior, recovery, and measured result.

Key Findings

- Application metrics: Implemented
- Prometheus integration: Implemented
- Grafana: Documented/evidenced
- CloudWatch alarms: Configured
- SNS notifications: Configured
- Cost budget: Implemented
- ECR lifecycle: Implemented
- HPA monitoring: Implemented
- Centralized application log shipping to CloudWatch: Not clearly implemented
- Chaos engineering: Partial

Suggested Improvements

Connect application logs explicitly to CloudWatch if CloudWatch logging is part of the requirement. Expand chaos experiments with a defined scenario, impact measurement, recovery validation, and result documentation.

Private & Confidential – Do not duplicate or distribute without written permission"

8. Documentation, Validation, and Production Readiness

Issue Identified

The documentation is extensive and professionally structured, but a few areas contain differences between documented architecture and the actual implementation.

Observed Behavior

The repository includes:

- README.
- Architecture documentation.
- Deployment guide.
- Getting Started guide.
- Troubleshooting guide.
- Cost optimization report.
- Production checklist.
- Branching strategy.
- Security documentation.
- Screenshots covering infrastructure and deployment stages.

The documentation provides useful evidence for EKS, ALB, Terraform, CI/CD, monitoring, security, and cost optimization.

However, some documentation claims are ahead of the actual configuration. For example, the architecture documentation states that the application uses IRSA to access Secrets Manager, while the Helm Deployment does not currently configure the required ServiceAccount. Similarly, the production workflow documentation describes a production deployment, while the actual workflow still references development ECR/cluster configuration.

Opportunity for Enhancement

The documentation should be synchronized with the final implementation after correcting the environment and secret-management configuration.

The production checklist is also primarily a checklist/template

Private & Confidential – Do not duplicate or distribute without written permission"

rather than evidence that every production-readiness item has actually been completed.

Why This Matters

In an enterprise project, documentation should describe the deployed system accurately. Otherwise, another engineer may follow the documentation and get a different result from what the repository actually implements.

Key Findings

- Repository documentation: Strong
- Architecture documentation: Detailed
- Deployment documentation: Detailed
- Troubleshooting documentation: Present
- Screenshots/evidence: Extensive
- Production checklist: Present
- Documentation-to-code consistency: Needs improvement in some areas

Suggested Improvements

After correcting the production CD configuration and IRSA/Secrets integration, update the corresponding documentation and screenshots. Clearly distinguish between implemented, configured but unused, and planned/optional components.

Private & Confidential – Do not duplicate or distribute without written permission"